smallwikiapp

A command that prints a random page of Wikipedia, in any language.

The tutorial builds the project around the command, one step at a time:

- packages, with **uv**
- one interface for every task, with `just`
- tests, and mocks
- a library, and an app that uses it
- format, lint, types, a git hook, CI
- documentation, made from the code

Steps and moves

A **step** is a tag, for example `step-00` or `step-01`.

A **move** is one small commit between two tags. It does one action, and a command at its end shows what it did.

Mode	What you do
Watch	**Show** checks out each move, and the notes explain the change
Do	You make each move from the notes, run the command at its end, and press **Done**

Every step starts from the code of the step before, so run `just setup` first.

The project before the code

```
uv init --package --python 3.13 --name wikiapp
```

File	What it says	Made by
`pyproject.toml`	the name, the Python, the dependencies, the command	`uv init`, then by hand
`.python-version`	which Python uv installs	`uv init`
`uv.lock`	the exact version of every package	`uv lock`
`justfile`	the tasks: `just` lists them	by hand

`just setup` makes `.venv` from the lockfile, on any machine.

A random page

```
https://en.wikipedia.org/api/rest_v1/page/random/summary
```

- **click** makes the command line
- **requests** asks Wikipedia
- the answer is JSON, with `title`, `extract` and many keys that wikiapp does not use

403 Forbidden

The code of 2024 no longer works. Wikipedia refuses a request whose User-Agent is a library's default, for example `python-requests/2.34.2`. The policy of Wikipedia asks for a User-Agent like the one below.

```
User-Agent: wikiapp/0.1.0 (https://github.com/rahuldave/smallwikiapp)
```

The program, its version, and where to find whoever runs it.

Tests, with mocks

A test that asks the real Wikipedia:

- fails on a train
- sees a different page each time

A **mock** replaces `requests.get` for the length of one test. It answers with a page that the test chooses, **Lorem Ipsum**.

Fixture	What a test receives
`runner`	click's CliRunner: runs the command, keeps what it printed
`mock_requests_get`	the mock: what it returns, and how it was called

Dependency groups

```
[dependency-groups]
test = ["pytest>=9.1.1", "pytest-cov>=7.1.0", "pytest-mock>=3.16.0"]
dev = [{ include-group = "test" }]
```

Someone who installs wikiapp gets click and requests, not pytest. `uv sync` installs `dev` by default.

A library and an app

File	Knows	Does
`wikipedia.py`	Wikipedia, requests	`random_page(language)`, `WikipediaError`
`console.py`	click	reads `--language`, prints, turns the error into a message

The library knows nothing of the command line. Another program could use it.

One error, at the edge

```
try:
    with requests.get(...) as response: ...
except requests.RequestException as error:
    raise WikipediaError(str(error)) from error
```

A user of the library catches one exception, not requests' many. The app turns it into `Error: ...` and exit code 1.

Format, lint, a hook

Tool	Recipe	What it does
ruff format	`just fmt`	one way to lay out the code; nothing to argue about
ruff check	`just lint`	rules: unused names, import order, security
validate-pyproject, uv lock --check, just --fmt --check	`just lint-config`	the configuration: pyproject.toml, the lockfile, the justfile
all of them, and the tests	`just check`	everything a commit must pass
pre-commit	git hook	runs `just check` before every commit

What the linter found

Code	Where	Fix
I001	imports out of order	`just lint --fix`
E402	an import below code	by hand
F841	a result never read, twice	by hand
S113	**a request with no timeout**	**by hand: the command could hang for ever**

Types

Tool	Asks
ruff, `ANN`	is a type written on every parameter and return?
ty, `just types`	can every call have the types the function asks for?

ty finds one problem:

```
importlib.metadata.version(__name__ or __package__)
#                          ^^^^^^^^^^^^^^^^^^^^^^^ str | None
```

The name of the package, `"wikiapp"`, is always a string.

End to end, and CI

- the two tests marked **e2e** ask the real Wikipedia, with `just test -m e2e`
- every other run leaves them out
- `requires-python = ">=3.11"`: the oldest Python still given security fixes

GitHub, on every push to main and every pull request:

	3.11	3.12	3.13	3.14
Linux	tests	tests	tests	tests
macOS	tests	tests	tests	tests
Windows	tests	tests	tests	tests

and one `just check`.

A page with a shape

```
class Page(BaseModel):
    title: str
    extract: str
```

- `Page.model_validate(answer)` keeps the two keys, and checks they are strings
- `page.title` is an attribute that ty and an editor know, and a misspelled name is an error
- an answer without the two keys raises `ValidationError`, which becomes `WikipediaError`

A language with a shape

```
API_URL = "https://{language}.wikipedia.org/..."
```

`--language 'example.com/x?'` would send the request to example.com.

```
LANGUAGE = re.compile(r"[a-z]+(-[a-z]+)*")   # en, fr, simple, zh-yue
```

Check what comes in, before it goes into an address.

Docs from the code

```
def random_page(
    language: str = "en",  # The language of the Wikipedia to ask, as its code: en, fr, es
) -> Page:  # A page chosen at random
    """Get a random page from Wikipedia.

    Raises WikipediaError when the language is not a language code, when the request fails, or
    when the answer is not a page.
    """
```

Part	From
signature, types	the code
what it does	the docstring
what each parameter or field is	its **docment**, the comment beside it

Written once, beside the code it describes.

smalldocs: one recipe, three uses

Command	Does
`just docs module wikipedia`	one module's reference, as Markdown
`just docs check`	each module without a docstring, and each function, class or field without its types and docs. It fails if any
`just docs`	the site, with the reference pages made from the code first

The check runs in `just check`, so the git hook refuses an undocumented function.

The docs on GitHub Pages

Each job on GitHub installs the group it needs, and no more.

Who	Installs	Runs
A person, `just setup`	`dev`: test, lint, docs	every recipe
GitHub, job `check`	`dev`	`just check`
GitHub, 12 jobs `tests`	the group `test`	`pytest`
GitHub, workflow `Docs`	the group `docs`	`smalldocs render docs`, then Pages

```
uv run --locked --no-default-groups --group test pytest
```

Repository	Its site
a fork	`https://OWNER.github.io/smallwikiapp/`
rahuldave/smallwikiapp	https://rahuldave.com/smallwikiapp/

The library as a package

```
smallwikiapp/
├── pyproject.toml            wikiapp: click, wikirandom
├── src/wikiapp/              the app
└── packages/wikirandom/
    ├── pyproject.toml        wikirandom: requests, pydantic
    └── src/wikirandom/       the library
```

A uv **workspace**: two packages, one lockfile, one `.venv`.

```toml
[tool.uv.sources]
wikirandom = { workspace = true }   # from packages/wikirandom, not from PyPI
```

Another program can install wikirandom without click.

Build

```
just build        # uv build --all-packages
```

File	What it is
`wikiapp-0.1.0.tar.gz`	the source archive
`wikiapp-0.1.0-py3-none-any.whl`	the wheel: what pip and uv install
`wikirandom-0.1.0...`	the same, for the library

- the metadata of the wheel names its dependencies and their ranges, for example `wikirandom>=0.1.0,<0.2`
- a licence, and a link to the repository, for each
- the version is in one place, `pyproject.toml`, and the code reads it from the installed package